



High-Performance E-Commerce Backend Engine

Django + PostgreSQL + Redis + Celery + Nginx	التقنية المستخدمة
https://github.com/ibrahimhana/high-performance-ecommerce-engine	رابط المشروع
نجاة الحلبي ابراهيم هنا سلمى جعفر محمد خالد عبد المجيد	أسماء الطلاب

1. بنية المشروع المختصرة

يعتمد المشروع على بنية Backend مبنية باستخدام Django، مع فصل واضح بين استقبال الطلبات، قاعدة البيانات، المهام الخلفية، وتوزيع الأحمال. الهدف من هذه البنية هو اختبار المتطلبات غير الوظيفية المطلوبة في المشروع بشكل عملي.

```
Client
↓
Nginx Load Balancer
↓
web1 / web2 / web3
↓
Django REST API
↓
PostgreSQL

Redis + Celery Worker: async background tasks
Celery Beat: scheduled batch jobs
```

2. الملفات المرتبطة بالاختبارات

الملف	الاستخدام في التقرير
scripts/race_condition_demo.py	تشغيل اختبار Race Condition وحماية المخزون.
docs/test-results/race_condition_result.txt	تؤخذ منه نتيجة اختبار التزامن والمخزون.
docs/test-results/capacity_web1.txt / capacity_web2.txt / capacity_web3.txt	تؤخذ منها نتائج Capacity Control وظهور 503 عند الضغط.
docs/test-results/celery_and_batch_logs.txt	تؤخذ منه نتائج Celery والـ Batch Processing.
docs/test-results/load_balancing_result.txt	تؤخذ منه نتائج توزيع الطلبات بين web1 و web2 و web3.
apps/orders/services.py	مكان تطبيق select for update و transaction.atomic.
apps/core/middleware.py	مكان تطبيق BoundedSemaphore للتحكم بالسعة.
apps/orders/tasks.py	مكان تطبيق Celery tasks و batch job.
nginx/nginx.conf	مكان تطبيق least conn load balancing.

اختبار 1: حماية المخزون من Race Condition

الهدف	التأكد من أن النظام لا يبيع كمية أكبر من المخزون المتاح عند وجود عدة عمليات checkout متزامنة على نفس المنتج.
الحل المطبق	استخدمنا transaction.atomic مع select_for_update لقفل صف المنتج أثناء تحديث المخزون، بحيث لا تستطيع عمليتان تعديل نفس المخزون في نفس الوقت.
مكان التطبيق في الكود	apps/orders/services.py scripts/race_condition_demo.py docs/test-results/race_condition_result.txt
الاستنتاج	إذا كان عدد الطلبات المقبولة لا يتجاوز كمية المخزون، فهذا يثبت أن قفل الصف داخل قاعدة البيانات يمنع Race Condition.

```
najat@MacBook-Air high-performance-ecommerce-engine % cat docs/test-results/race_condition_result.txt
-> logging in as 'demo' on http://localhost:8080
-> product RACE-001 id=3 starting_stock=31

=== Phase 1: UNSAFE checkout (proving the race exists) ===
mode                UNSAFE (no lock)
requests_fired       100
successful_sales (HTTP 201) 100
out_of_stock_rejected (HTTP 409) 0
other_errors         0
stock_before         50
stock_after          43
consistent           False

=== Phase 2: SAFE checkout (with row lock) ===
mode                SAFE (SELECT FOR UPDATE)
requests_fired       100
successful_sales (HTTP 201) 50
out_of_stock_rejected (HTTP 409) 50
other_errors         0
stock_before         50
stock_after          0
consistent           True

Interpretation:
* Phase 1 should show successes != 50 OR stock_after != 0
  (oversell or inconsistent state -> classic race condition).
* Phase 2 should show exactly 50 successes, 50 conflicts,
  stock_after = 0, consistent = True.
```

اختبار 2: إدارة الموارد والتحكم بالسعة

الهدف	التأكد من أن النظام لا يسمح بعدد غير محدود من الطلبات الثقيلة في نفس الوقت، خصوصاً عمليات checkout.
الحل المطبق	استخدمنا BoundedSemaphore داخل CapacityControlMiddleware لتحديد عدد الطلبات الثقيلة المسموح بها في الوقت نفسه. عند امتلاء السعة، يرجع النظام 503 بدل الانهيار.
مكان التطبيق في الكود	apps/core/middleware.py docs/test-results/capacity_web1.txt docs/test-results/capacity_web2.txt docs/test-results/capacity_web3.txt
الاستنتاج	ظهور 503 عند الضغط يعني أن النظام يحمي موارده بدل أن يسمح لكل الطلبات بالدخول إلى قاعدة البيانات.

تظهر النتيجة أن CapacityControlMiddleware تم تشغيله داخل web1، وأن النظام قام بإنشاء semaphore بسعة 8 لكل process للتحكم بعدد الطلبات الثقيلة المتزامنة. في هذا التشغيل لم تظهر استجابات 503 لأن الضغط لم يتجاوز الحد المحدد، لكن ظهور قيمة cap يثبت أن آلية التحكم بالسعة مفعلة داخل النظام.

```

majat@MacBe ~ % rformance-e-commerce-engine % cat docs/test-results/capacity_web1.txt
hpe_web1 Output (OSU) 2:30:30.094] INFO apps.core inst=web1 :: POST /api/orders/checkout-direct/ 165.6ms
hpe_web1 Conflict: /api/orders/checkout-direct/
hpe_web1 [2026-05-17 22:30:30.094] WARNING django.request inst=web1 :: Conflict: /api/orders/checkout-direct/
hpe_web1 172.18.0.9 - - [17/May/2026:22:30:30 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 409 43 "-" "python-requests/2.32.5"
hpe_web1 [2026-05-17 22:30:30.096] INFO apps.core inst=web1 :: POST /api/orders/checkout-direct/ 60.8ms
hpe_web1 Conflict: /api/orders/checkout-direct/
hpe_web1 [2026-05-17 22:30:30.096] WARNING django.request inst=web1 :: Conflict: /api/orders/checkout-direct/
hpe_web1 172.18.0.9 - - [17/May/2026:22:30:30 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 409 43 "-" "python-requests/2.32.5"
hpe_web1 [2026-05-17 22:30:30.097] INFO apps.core inst=web1 :: POST /api/orders/checkout-direct/ 163.6ms
hpe_web1 Conflict: /api/orders/checkout-direct/
hpe_web1 [2026-05-17 22:30:30.097] WARNING django.request inst=web1 :: Conflict: /api/orders/checkout-direct/
hpe_web1 172.18.0.9 - - [17/May/2026:22:30:30 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 409 43 "-" "python-requests/2.32.5"
hpe_web1 [2026-05-17 22:30:30.101] INFO apps.core inst=web1 :: POST /api/orders/checkout-direct/ 32.4ms
hpe_web1 Conflict: /api/orders/checkout-direct/
hpe_web1 [2026-05-17 22:30:30.101] WARNING django.request inst=web1 :: Conflict: /api/orders/checkout-direct/
hpe_web1 172.18.0.9 - - [17/May/2026:22:30:30 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 409 43 "-" "python-requests/2.32.5"
hpe_web1 [2026-05-17 22:30:30.103] INFO apps.core inst=web1 :: POST /api/orders/checkout-direct/ 30.4ms
hpe_web1 Conflict: /api/orders/checkout-direct/
hpe_web1 [2026-05-17 22:30:30.103] WARNING django.request inst=web1 :: Conflict: /api/orders/checkout-direct/
hpe_web1 172.18.0.9 - - [17/May/2026:22:30:30 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 409 43 "-" "python-requests/2.32.5"
hpe_web1 [2026-05-17 22:30:30.108] INFO apps.core inst=web1 :: POST /api/orders/checkout-direct/ 33.5ms
hpe_web1 Conflict: /api/orders/checkout-direct/
hpe_web1 [2026-05-17 22:30:30.108] WARNING django.request inst=web1 :: Conflict: /api/orders/checkout-direct/
hpe_web1 172.18.0.9 - - [17/May/2026:22:30:30 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 409 43 "-" "python-requests/2.32.5"
hpe_web1 [2026-05-17 22:30:30.114] INFO apps.core inst=web1 :: POST /api/orders/checkout-direct/ 20.1ms
hpe_web1 Conflict: /api/orders/checkout-direct/
hpe_web1 [2026-05-17 22:30:30.114] WARNING django.request inst=web1 :: Conflict: /api/orders/checkout-direct/
hpe_web1 172.18.0.9 - - [17/May/2026:22:30:30 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 409 43 "-" "python-requests/2.32.5"
hpe_web1 [2026-05-17 22:30:30.115] INFO apps.core inst=web1 :: POST /api/orders/checkout-direct/ 19.2ms
hpe_web1 Conflict: /api/orders/checkout-direct/
hpe_web1 [2026-05-17 22:30:30.115] WARNING django.request inst=web1 :: Conflict: /api/orders/checkout-direct/
hpe_web1 172.18.0.9 - - [17/May/2026:22:30:30 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 409 43 "-" "python-requests/2.32.5"
hpe_web1 [2026-05-17 22:30:30.115] INFO apps.core inst=web1 :: GET /api/health/ 1.5ms
hpe_web1 172.18.0.9 - - [17/May/2026:22:31:14 +0000] "GET /api/health/ HTTP/1.1" 200 43 "-" "curl/8.7.1"
hpe_web1 [2026-05-17 22:31:14.963] INFO apps.core inst=web1 :: GET /api/health/ 1.4ms
hpe_web1 172.18.0.9 - - [17/May/2026:22:31:14 +0000] "GET /api/health/ HTTP/1.1" 200 43 "-" "curl/8.7.1"
hpe_web1 [2026-05-17 22:31:14.980] INFO apps.core inst=web1 :: GET /api/health/ 1.3ms
hpe_web1 172.18.0.9 - - [17/May/2026:22:31:14 +0000] "GET /api/health/ HTTP/1.1" 200 43 "-" "curl/8.7.1"
hpe_web1 [entrypoint] waiting for Postgres to be ready ...
hpe_web1 [entrypoint] Postgres reachable.
hpe_web1 [2026-05-18 15:53:03 +0000] [1] [INFO] Starting unicorn 22.0.0
hpe_web1 [2026-05-18 15:53:03 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
hpe_web1 [2026-05-18 15:53:03 +0000] [1] [INFO] Using worker: gthread
hpe_web1 [2026-05-18 15:53:03 +0000] [8] [INFO] Booting worker with pid: 8
hpe_web1 [2026-05-18 15:53:03 +0000] [9] [INFO] Booting worker with pid: 9
hpe_web1 [2026-05-18 15:53:03 +0000] [10] [INFO] Booting worker with pid: 10
hpe_web1 [2026-05-18 15:53:03.460] INFO apps.core inst=web1 :: CapacityControl: per-process semaphore cap=8
hpe_web1 [2026-05-18 15:53:03.524] INFO apps.core inst=web1 :: CapacityControl: per-process semaphore cap=8
hpe_web1 [2026-05-18 15:53:03.586] INFO apps.core inst=web1 :: CapacityControl: per-process semaphore cap=8

```

تظهر النتيجة أن النظام أعاد استجابات 503 Service Unavailable عند إرسال عدة طلبات checkout متزامنة بعد تخفيض MAX_CONCURRENT_HEAVY_REQUESTS للاختبار. هذا يثبت أن CapacityControlMiddleware قام برفض الطلبات الزائدة بدل السماح لها باستهلاك موارد النظام.

```

docs/test-results/capacity_web1.txt:hpe_web1 | 172.18.0.9 - - [18/May/2026:16:07:
14 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 503 47 "-" "python-requests
/2.32.5"
docs/test-results/capacity_web1.txt:hpe_web1 | 172.18.0.9 - - [18/May/2026:16:07:
14 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 503 47 "-" "python-requests
/2.32.5"
docs/test-results/capacity_web1.txt:hpe_web1 | Service Unavailable: /api/orders/c
heckout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | [2026-05-18 16:07:14,143] ERROR dj
ango.request inst=web1 :: Service Unavailable: /api/orders/checkout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | 172.18.0.9 - - [18/May/2026:16:07:
14 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 503 47 "-" "python-requests
/2.32.5"
docs/test-results/capacity_web1.txt:hpe_web1 | Service Unavailable: /api/orders/c
heckout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | [2026-05-18 16:07:14,148] ERROR dj
ango.request inst=web1 :: Service Unavailable: /api/orders/checkout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | 172.18.0.9 - - [18/May/2026:16:07:
14 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 503 47 "-" "python-requests
/2.32.5"
docs/test-results/capacity_web1.txt:hpe_web1 | Service Unavailable: /api/orders/c
heckout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | [2026-05-18 16:07:14,159] ERROR dj
ango.request inst=web1 :: Service Unavailable: /api/orders/checkout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | 172.18.0.9 - - [18/May/2026:16:07:
14 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 503 47 "-" "python-requests
/2.32.5"
docs/test-results/capacity_web1.txt:hpe_web1 | Service Unavailable: /api/orders/c
heckout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | [2026-05-18 16:07:14,162] ERROR dj
ango.request inst=web1 :: Service Unavailable: /api/orders/checkout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | 172.18.0.9 - - [18/May/2026:16:07:
14 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 503 47 "-" "python-requests
/2.32.5"
docs/test-results/capacity_web1.txt:hpe_web1 | Service Unavailable: /api/orders/c
heckout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | [2026-05-18 16:07:14,163] ERROR dj
ango.request inst=web1 :: Service Unavailable: /api/orders/checkout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | 172.18.0.9 - - [18/May/2026:16:07:
14 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 503 47 "-" "python-requests
/2.32.5"
docs/test-results/capacity_web1.txt:hpe_web1 | Service Unavailable: /api/orders/c
heckout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | [2026-05-18 16:07:14,164] ERROR dj
ango.request inst=web1 :: Service Unavailable: /api/orders/checkout-direct/
docs/test-results/capacity_web1.txt:hpe_web1 | 172.18.0.9 - - [18/May/2026:16:07:
14 +0000] "POST /api/orders/checkout-direct/ HTTP/1.1" 503 47 "-" "python-requests
/2.32.5"
docs/test-results/capacity_web1.txt:hpe_web1 | Service Unavailable: /api/orders/c
heckout-direct/

```

اختبار 3: المعالجة غير المتزامنة باستخدام Celery

الهدف

التأكد من أن المهام الثانوية مثل الفاتورة والإشعارات لا يتم تنفيذها داخل request الأساسي الخاص بعملية checkout.

الحل المطبق

استخدمنا Celery مع Redis. بعد إنشاء الطلب، يتم إرسال send_order_notifications و send_invoice_email إلى Celery queue باستخدام delay، ويعود response للمستخدم بدون انتظار تنفيذ هذه المهام.

apps/orders/tasks.py config/celery.py docker-compose.yml docs/test-results/celery_and_batch_logs.txt	مكان التطبيق في الكود
أظهر في screenshot أسماء المهام مثل send_invoice_email أو send_order_notifications، ويفضل أن يظهر received أو succeeded.	النتيجة التي يجب إظهارها
ظهور مهام Celery في logs بعد تنفيذ checkout يثبت أن المهام الثانوية انتقلت إلى background worker ولم تبق داخل request الرئيسي.	الاستنتاج

```
najat@MacBook-Air high-performance-ecommerce-engine % cat docs/test-results/celery_and_batch_logs.txt
hpe_celery_worker | [2026-05-17 22:30:49,251: INFO/ForkPoolWorker-2] Task apps.orders.tasks.send_order_notifications[4deb3c7f-273d-4167-be30-90db899a5835] succeeded in 0.2015746669994769s: None
hpe_celery_worker | [2026-05-17 22:30:49,253: INFO/MainProcess] Task apps.orders.tasks.send_order_notifications[14e550aa-ab7b-4134-927a-99567135cf2c] received
hpe_celery_worker | [2026-05-17 22:30:49,274] INFO apps.orders.tasks inst=celery :: invoice generated for order=376 user=1 total=10.00
hpe_celery_worker | [2026-05-17 22:30:49,274] INFO apps.core inst=celery :: timed [task.send_invoice_email] 258.18ms
hpe_celery_worker | [2026-05-17 22:30:49,275: INFO/ForkPoolWorker-4] Task apps.orders.tasks.send_invoice_email[6205a0f3-9dd7-4c6c-90bc-c7b499efdfdb] succeeded in 0.25923941699966235s: None
hpe_celery_worker | [2026-05-17 22:30:49,276: INFO/MainProcess] Task apps.orders.tasks.send_invoice_email[ec8a8c0b-af0e-48fc-9cd9-43c12eed23e4] received
```

اختبار 4: معالجة المبيعات اليومية على دفعات

التأكد من أن النظام يستطيع تنفيذ جرد المبيعات اليومية كـ background job بدون تحميل كل البيانات في الذاكرة دفعة واحدة.	الهدف
استخدمنا rollup_daily_sales كـ Celery task، وتتم قراءة OrderItem باستخدام chunk_size=500 (iterator) لمعالجة البيانات على دفعات.	الحل المطبق
apps/orders/tasks.py config/celery.py docs/test-results/celery_and_batch_logs.txt	مكان التطبيق في الكود

الاستنتاج

ظهر تقرير المبيعات اليومية يثبت أن الـ batch job عمل بنجاح، وأن معالجة البيانات تمت في الخلفية وبأسلوب chunks.

```
najat@MacBook-Air high-performance-ecommerce-engine % curl http://localhost:8080/api/orders/reports/daily/
[{"id":1,"date":"2026-05-17","orders_count":150,"units_sold":150,"gross_revenue":"1500.00","generated_at":"2026-05-17T21:53:01.845300Z"}]
```

```
najat@MacBook-Air high-performance-ecommerce-engine % docker compose logs celery_worker | grep "rollup_daily_sales\|daily\|chunk\|report"
hpe_celery_worker | . apps.orders.tasks.rollup_daily_sales
najat@MacBook-Air high-performance-ecommerce-engine %
```

اختبار 5: توزيع الأحمال بين أكثر من خادم

الهدف	التأكد من أن Nginx يوزع الطلبات على أكثر من نسخة من Django بدل إرسال كل الطلبات إلى خادم واحد.
الحل المطبق	استخدمنا Nginx أمام ثلاث نسخ web1 و Django: web2 و web3، مع استراتيجية least_conn لتوجيه الطلب إلى الخادم الأقل انشغالاً.
مكان التطبيق في الكود	nginx/nginx.conf docker-compose.yml docs/test-results/load_balancing_result.txt
الاستنتاج	ظهر أكثر من instance في النتائج يثبت أن Nginx يوزع الطلبات ولا يعتمد على خادم واحد فقط.


```
npe_celery_worker | ... apps.orders.tasks:Rollup_daily_sales
● najat@MacBook-Air high-performance-ecommerce-engine % for i in {1..12}; do
  curl -s -D - -o /dev/null http://localhost:8080/api/health/ | grep -E "X-I
instance|X-Served-By"
done
X-Instance: web1
X-Served-By: 172.18.0.4:8000
X-Instance: web1
X-Served-By: 172.18.0.4:8000
X-Instance: web1
X-Served-By: 172.18.0.4:8000
X-Instance: web1
X-Served-By: 172.18.0.4:8000
X-Instance: web1
X-Served-By: 172.18.0.4:8000
X-Instance: web1
X-Served-By: 172.18.0.4:8000
X-Instance: web1
X-Served-By: 172.18.0.4:8000
X-Instance: web1
X-Served-By: 172.18.0.4:8000
X-Instance: web1
X-Served-By: 172.18.0.4:8000
X-Instance: web2
X-Served-By: 172.18.0.6:8000
X-Instance: web3
X-Served-By: 172.18.0.7:8000
X-Instance: web1
X-Served-By: 172.18.0.4:8000
○ najat@MacBook-Air high-performance-ecommerce-engine %
```